Leitura — Automação e fluxo de valor: DevOps, CI, containers, DORA e VSM (Sprint 2)

Guia de apoio para as duas aulas da Sprint 2: DevOps, integração contínua e containers (26/10) e métricas DORA e mapeamento do fluxo de valor (28/10, presencial). Na Sprint 1, a pergunta era "como o trabalho flui, e onde ele trava". Nesta, é "como automatizar a entrega, e como saber, com números, se o processo está melhorando".

Os slides das aulas são `slides/processos-slides-08.md` e `slides/processos-slides-09.md`. Os números do MUSI foram coletados do repositório em 15/10/2026, com os comandos mostrados.

Como ler: os capítulos 2 a 5, de 26/10; os capítulos 6 e 7, de 28/10. O capítulo 8 junta tudo na entrega.

Capítulos:

1. Onde a Sprint 2 se encaixa
2. DevOps: CALMS e os Três Caminhos
3. Integração contínua
4. A proteção da `main`
5. Containers e `docker compose`
6. Métricas DORA
7. Mapeamento do fluxo de valor
8. A entrega da Sprint 2
9. Exercícios e dúvidas frequentes

1. Onde a Sprint 2 se encaixa

A Sprint 2 é a última do semestre: o que for entregue em 30/11 é o produto final. Três dos cinco critérios da rubrica são o assunto desta leitura:

Critério	Peso	Capítulos
Pipeline de CI e containerização	25%	3, 4 e 5
Métricas DORA	20%	6
Mapeamento de Fluxo de Valor	25%	7
MVP finalizado	15%	—
Retrospectiva final	15%	8

As três partes se apoiam. O pipeline automatiza a entrega; as métricas DORA medem como a entrega se comporta; o mapa do fluxo mostra onde o trabalho espera antes de chegar ao pipeline. A retrospectiva

final usa os números das três para mostrar como o processo mudou.

2. DevOps: CALMS e os Três Caminhos

2.1 O problema

Em muitas organizações, quem escreve o código e quem o põe no ar eram equipes diferentes. O desenvolvimento era cobrado por entregar mudanças; a operação, por manter o sistema estável. Para a operação, cada mudança era um risco, e a forma de reduzir o risco era mudar pouco. O resultado eram entregas raras e grandes, cada uma um evento arriscado, com muitas mudanças juntas e difíceis de desfazer.

DevOps é a proposta de tratar entrega e operação como responsabilidade de uma equipe só, apoiada em automação e em medição. Não é um cargo nem uma ferramenta: é um conjunto de práticas e uma cultura.

2.2 CALMS

CALMS é uma forma de lembrar as cinco dimensões de uma adoção de DevOps:

Letra	Dimensão	O que significa	Onde já apareceu na disciplina
C	Cultura	responsabilidade compartilhada; falha tratada como aprendizado, sem culpados	a retrospectiva
A	Automação	o que se repete vira script ou pipeline	o CI da Sprint 0
L	Lean	fluxo, lotes pequenos, menos desperdício	Lean e Kanban, na Sprint 1
M	Medição	decidir com dados	as métricas de fluxo
S	Compartilhamento	conhecimento e ferramentas abertos à equipe	README, ADRs, revisão de código

📖 Ref. Atlassian — CALMS framework: https://www.atlassian.com/devops/frameworks/calms-framework

2.3 Os Três Caminhos

Os Três Caminhos, de Gene Kim, descrevem os princípios por trás das práticas de DevOps:

- **Primeiro Caminho — fluxo.** O trabalho anda da esquerda (a ideia) para a direita (o usuário) sem paradas: lotes pequenos, limite de WIP, integração frequente. É o que o Lean e o Kanban trabalharam na Sprint 1.
- **Segundo Caminho — feedback.** A informação volta da direita para a esquerda, rápido: testes automatizados, monitoramento, revisão. Um problema descoberto minutos depois da mudança é

barato de corrigir; descoberto semanas depois, é caro.
- **Terceiro Caminho — aprendizado contínuo.** Experimentar, errar e aprender com o erro, todo dia; reservar tempo para melhorar o próprio processo. É o papel da retrospectiva e das ações com dono, prazo e verificação.

📖 Ref. IT Revolution — The Three Ways: https://itrevolution.com/articles/the-three-ways-principles-underpinning-devops/

3. Integração contínua

3.1 O que é

Integração contínua é a prática de cada integrante integrar o seu trabalho à linha principal (a `main`) com frequência, pelo menos uma vez por dia, com cada integração verificada por um build automatizado que inclui os testes.

As duas partes importam. Integrar com frequência sem verificar só espalha problemas mais depressa. Verificar sem integrar com frequência deixa os ramos divergirem por dias, e a integração volta a ser um evento difícil, perto da entrega.

Sem integração contínua	Com integração contínua
cada um trabalha dias num ramo próprio	ramos curtos, integrados em horas ou poucos dias
a integração é um evento, perto da entrega	a integração é rotina
"na minha máquina funciona"	funciona numa máquina limpa, a do CI
o problema aparece semanas depois	o problema aparece minutos depois do push

📖 Ref. Martin Fowler — Continuous Integration: https://martinfowler.com/articles/continuousIntegration.html

📖 Ref. DORA — Continuous integration: https://dora.dev/capabilities/continuous-integration/

3.2 O que o CI deve rodar

A rubrica pede build, testes e lint, em todo push e em todo pull request:

Etapa	Pergunta que responde
Build	o código compila numa máquina limpa, só com o que está no repositório?
Testes	o comportamento continua certo?
Lint	o código segue as regras de estilo combinadas?

Duas qualidades fazem o CI ser útil, e não um obstáculo:

- **Rápido.** O CI é feedback. Se demora meia hora, a equipe para de esperar por ele e integra mesmo assim. Minutos é o tempo certo.
- **Igual à máquina de quem desenvolve.** Os comandos do CI são os mesmos que o grupo roda localmente (`make test`, `./gradlew test`, `mise run test`). Um passo que só o CI conhece vira uma falha que ninguém consegue reproduzir.

3.3 Anatomia de um workflow do GitHub Actions

```yaml
name: ci                              # o nome do workflow

on:                                   # quando roda
  push:
    branches: [main]
  pull_request:

jobs:
  verificar:                          # o nome do job: é o status check que aparece no PR
    runs-on: ubuntu-latest            # uma máquina limpa, nova a cada execução
    steps:
      - uses: actions/checkout@v5     # baixa o código do commit
      - run: make build
      - run: make lint
      - run: make test                # se um comando falha, o job fica vermelho
```

- `on` define os eventos. `pull_request` faz o CI rodar no PR, antes do merge, que é quando ele mais ajuda.
- Cada `job` roda numa máquina nova. Os `steps` rodam em sequência; um que falha interrompe o job.
- `uses` reaproveita uma ação publicada (baixar o código, instalar uma linguagem); `run` executa um comando de terminal.

📖 Ref. GitHub Actions — Quickstart: https://docs.github.com/en/actions/get-started/quickstart

4. A proteção da `main`

4.1 Por que o CI sozinho não basta

Um CI que só avisa não impede nada: um PR vermelho continua com o botão de merge disponível, e um push direto na `main` nem passa por PR. O que impede é a regra de proteção da branch, que transforma

o aviso em bloqueio.

Regra	Efeito
Exigir pull request antes do merge	ninguém faz push direto na `main`
Exigir status checks	o merge fica bloqueado enquanto o job do CI não estiver verde
Exigir revisão aprovada (opcional)	outra pessoa precisa aprovar o PR
Incluir administradores	a regra vale para todos, inclusive para quem é dono do repositório

No GitHub, a configuração fica em *Settings → Branches* (proteção clássica) ou em *Settings → Rules → Rulesets*. O status check exigido é escolhido pelo nome do job do workflow, e só aparece na lista depois de o workflow ter rodado pelo menos uma vez.

📖 Ref. GitHub — About protected branches: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

4.2 Conferir pelo terminal

O `gh api` chama a API REST do GitHub para ler a configuração de proteção da branch, e o `--jq` filtra o JSON e mostra só o que interessa: os status checks exigidos, se o PR é obrigatório e se a regra vale para administradores.

```
gh api repos/fmarquesfilho/musi/branches/main/protection \
  --jq '{checks: .required_status_checks.contexts, pr: (.required_pull_request_reviews !=
        null), admins: .enforce_admins.enabled}'
```

```
{"admins":false,"checks":["ci"],"pr":false}
```

No MUSI, o CI é obrigatório, mas o PR não, e o dono do repositório pode passar por cima. É a ação A2 da retrospectiva da Sprint 1, cumprida em parte.

4.3 Provar que a proteção funciona

A rubrica pede uma proteção "impedindo merge com falha". A forma de mostrar é testá-la:

1. Num ramo, quebrar um teste de propósito e abrir um PR.
2. Ver o check do CI ficar vermelho e o merge bloqueado, com a mensagem de que o status check exigido não passou.
3. Corrigir, fazer push no mesmo ramo, e ver o check rodar de novo e ficar verde.

Os três comandos abaixo conferem isso pelo terminal: o estado dos checks do PR, as últimas execuções e o log do passo que falhou.

```
gh pr checks              # o estado dos checks do PR atual
gh run list --limit 5     # as últimas execuções
gh run view --log-failed  # só o log do passo que falhou
```

Armadilhas comuns: o workflow não roda em `pull_request` (e o check nunca aparece no PR); o nome do job mudou e a regra continua exigindo o nome antigo; os administradores ficaram fora da regra.

5. Containers e `docker compose`

5.1 O problema que resolvem

Um projeto que roda numa máquina e não roda em outra, por diferença de versão da linguagem, do banco ou de uma biblioteca do sistema, é um caso de "na minha máquina funciona". Um container empacota o programa com tudo de que ele precisa para rodar, e roda do mesmo jeito em qualquer máquina com Docker.

Termo	O que é
Imagem	um pacote com o programa e o sistema de arquivos de que ele precisa
Container	uma imagem em execução, isolada dos outros processos
`Dockerfile`	a receita da imagem, em texto, versionada com o código
Registro	onde as imagens ficam para outras máquinas baixarem (Docker Hub, GHCR)

```
FROM eclipse-temurin:25-jre           # a imagem de partida
COPY build/install/app /app           # o que entra na imagem
EXPOSE 8080                           # a porta que o programa usa
CMD ["/app/bin/app"]                  # o comando que roda quando o container inicia
```

A primeira instrução (`FROM`) escolhe a imagem de partida; `COPY` leva o programa para dentro; `EXPOSE` declara, sem abrir, a porta; `CMD` é o comando que roda quando o container inicia.

Cada instrução do `Dockerfile` vira uma **camada**, e as camadas são reusadas em cache: mudou só o código, as dependências baixadas antes ficam em cache. Uma imagem **multi-stage** compila num primeiro estágio e copia só o binário para a imagem final, que fica mínima — o compilador não vai junto.

📖 Ref. Docker — multi-stage builds: https://docs.docker.com/build/building/multi-stage/

📖 Ref. Docker — What is a container?: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

5.2 O sistema inteiro com um comando

Um sistema real tem mais de um processo: a aplicação, o banco, às vezes outros serviços. O `docker compose` descreve todos num arquivo e sobe tudo com um comando:

```yaml
services:
  banco:
    image: postgres:17-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U tarefas"]
  api:
    build: .
    ports: ["8080:8080"]
    environment:
      DB_URL: jdbc:postgresql://banco:5432/tarefas   # dentro do compose, o nome é o endereço
    depends_on:
      banco: { condition: service_healthy }          # a API só sobe com o banco pronto
```

- Cada serviço vira um container, e todos ficam numa rede própria, onde o nome do serviço é o endereço (`banco:5432`).
- O `healthcheck` diz como saber se o serviço está pronto; o `depends_on` com `service_healthy` faz a ordem de subida respeitar isso.
- "Do zero" quer dizer numa máquina que só tem Docker: sem a linguagem instalada, sem banco configurado à mão, sem passos manuais.

O MUSI (`docker-compose.yml`) sobe banco, duas APIs (Ktor e Quarkus) e um serviço em Go com `docker compose up --build`.

📖 Ref. Docker Compose: https://docs.docker.com/compose/

5.3 O que muda na definição de pronto

Antes	Na Sprint 2
funciona na máquina de quem fez	o CI está verde no PR
alguém olhou o código	o PR foi revisado e só integrado com o CI verde
o README explica como instalar	`docker compose up` sobe o projeto do zero

6. Métricas DORA

6.1 Por que medir a entrega

O programa de pesquisa DORA (DevOps Research and Assessment) estuda há mais de uma década o desempenho de entrega de software de milhares de equipes. O resultado central contraria a intuição: velocidade e estabilidade não competem. Equipes que entregam com mais frequência também tendem a falhar menos e a se recuperar mais rápido, porque entregam em lotes pequenos, mais fáceis de testar, revisar e desfazer.

Por isso as métricas vêm em dois grupos, vazão e instabilidade, e uma equipe olha os dois ao mesmo tempo.

6.2 As cinco métricas

Grupo	Métrica	Definição
Vazão	Tempo de espera da mudança (*change lead time*)	do commit no controle de versão até a mudança rodar em produção
Vazão	Frequência de implantação (*deployment frequency*)	quantas implantações por período, ou o intervalo entre elas
Vazão	Tempo de recuperação (*failed deployment recovery time*)	quanto leva para recuperar de uma implantação que falhou e exigiu intervenção imediata
Instabilidade	Taxa de falha de mudança (*change fail rate*)	fração das implantações que exigiram intervenção imediata, como reverter ou corrigir às pressas
Instabilidade	Taxa de retrabalho (*deployment rework rate*)	fração das implantações não planejadas, feitas por causa de um incidente em produção

Das cinco, duas são **taxas** (falha e retrabalho: a fração dos eventos que deram errado) e duas são **intervalos** (tempo de espera e recuperação: a duração entre dois instantes); a frequência pode ser uma contagem por período ou o intervalo entre implantações. Essa diferença muda a coleta: uma taxa conta eventos, um intervalo subtrai datas.

A rubrica pede ao menos três, com o método de coleta descrito e reprodutível, e a interpretação.

📖 Ref. DORA — Software delivery performance metrics: https://dora.dev/guides/dora-metrics/

6.3 Primeiro, o que é uma implantação

Nenhuma das cinco métricas tem número sem uma definição de implantação. Cada grupo escolhe a sua, conforme o projeto, e a escreve:

Se o projeto...	Uma implantação é
publica automaticamente a cada push na `main` (Render, Vercel)	cada push na `main`
publica imagens pelo CI	cada execução bem-sucedida do job de publicação
gera releases ou tags	cada release ou tag
ainda não está no ar	cada merge na `main` com o CI verde (e o documento diz isso)

O método começa por uma frase: "Neste projeto, uma implantação é ___, e a contamos com o comando ___."

6.4 Coletar do repositório

Os comandos abaixo leem, no repositório, os eventos crus que alimentam as métricas: as execuções do CI na `main`, os tempos de cada PR, o resultado de cada execução e os commits por semana. O GitHub guarda essas datas e conclusões; o `gh` e o `git` resumem, sem a equipe precisar anotar nada à mão.

```
# Execuções do CI na main, com data e resultado
gh run list --branch main --workflow ci.yml --limit 100 --json createdAt,conclusion


# Da abertura ao merge de cada PR, em minutos
gh pr list --state merged --limit 100 --json number,createdAt,mergedAt \
  --jq '.[] | "#\(.number): \(((.mergedAt|fromdate) - (.createdAt|fromdate)) / 60 | floor)
        min"'


# Execuções na main agrupadas por resultado
gh run list --branch main --workflow ci.yml --limit 100 --json conclusion \
  --jq 'group_by(.conclusion) | map({(.[0].conclusion): length}) | add'


# Commits na main por semana
git log origin/main --format=%ad --date=format:%G-W%V | sort | uniq -c
```

Métrica	Fonte possível	Limitação a declarar
Frequência de implantação	execuções do CI na `main`, releases, log da plataforma	o CI conta tentativas, inclusive as que falharam
Tempo de espera da mudança	PRs, da abertura ao merge (ou do primeiro commit ao merge)	a abertura do PR não é o primeiro commit
Taxa de falha de mudança	execuções vermelhas na `main`; reversões	só capta as falhas que o CI ou a equipe registram

Métrica	Fonte possível	Limitação a declarar
Tempo de recuperação	da execução vermelha na `main` à próxima verde	só para falhas que o CI detecta
Taxa de retrabalho	implantações marcadas como correção de incidente	exige registrar o motivo de cada implantação

Um método reprodutível é aquele que outra pessoa consegue repetir e chegar ao mesmo número: a definição de implantação, os comandos exatos, o período e o que foi descartado.

6.5 Um exemplo: o MUSI

No MUSI, cada push na `main` dispara uma nova versão no Render, então uma implantação é um push na `main`, contado pelas execuções do CI. Período: 26/08 a 04/10/2026.

Métrica	Valor	Cálculo
Frequência	18 implantações em 6 semanas; 15 delas na primeira semana	19 execuções do CI na `main`, menos uma cancelada
Tempo de espera da mudança	36 e 48 min	os dois PRs integrados, da abertura ao merge
Taxa de falha	4 de 18, cerca de 22%	execuções vermelhas sobre o total concluído
Tempo de recuperação	11 min, 13 min e cerca de 26 h	de cada execução vermelha à próxima verde

A interpretação é a parte que dá sentido aos números:

- **A média esconde o padrão.** Três implantações por semana, em média, descreveria mal o projeto: houve uma semana intensa de montagem e, depois, quase nada. Mostrar a distribuição por semana diz mais que a média.
- **As falhas se concentram no início.** Três das quatro foram na primeira semana, quando o próprio CI estava sendo montado.
- **Tempo de espera curto não é necessariamente bom.** Os dois PRs foram integrados em menos de uma hora porque não tiveram revisão. O número melhora quando se pula uma etapa.
- **Uma recuperação de 26 horas** mostra que a falha atravessou a noite sem ninguém olhando. Não é um problema técnico: é a ausência de alguém responsável por manter a `main` verde.

6.6 Cuidados

- **Medir o sistema, não as pessoas.** As métricas descrevem como o processo entrega. Usadas para avaliar indivíduos, deixam de ser confiáveis.

- **Não comparar equipes.** Produtos, tamanhos e tecnologias diferentes tornam a comparação enganosa. A comparação útil é da equipe com ela mesma, ao longo do tempo.
- **Não transformar em meta.** Quando um número vira meta, a equipe aprende a mexer no número (lei de Goodhart): PRs vazios aumentam a frequência sem entregar nada.
- **Olhar a tendência.** O valor de uma semana diz pouco; a mudança ao longo da sprint diz mais.
- **Sempre com o método.** Um número sem "como foi obtido" não pode ser conferido, e não conta na rubrica.

7. Mapeamento do fluxo de valor

7.1 O que é

O mapeamento do fluxo de valor (*value stream mapping*, VSM) vem do Lean. Consiste em desenhar o caminho que um pedido percorre, da ideia até chegar ao usuário, e medir quanto tempo ele passa em cada etapa, separando o tempo em que alguém trabalha nele do tempo em que ele espera.

Termo	O que é
Tempo de processamento	o tempo em que alguém está de fato trabalhando no item
Tempo de espera	o tempo em que o item está parado, numa fila, esperando alguém
Lead time	do pedido à entrega: a soma de todos os tempos de processamento e de espera
Eficiência de fluxo	tempo de processamento total ÷ lead time × 100
Gargalo	a etapa que limita o fluxo inteiro: onde o trabalho mais se acumula ou espera

O mapa mostra o que o quadro esconde. Um cartão em "Em revisão" pode estar sendo revisado ou pode estar parado há dois dias; no quadro, os dois casos parecem iguais.

📖 Ref. Lean Enterprise Institute — Value-stream mapping: https://www.lean.org/lexicon-terms/value-stream-mapping/

7.2 Um exemplo com números

Uma história de usuário de um projeto da disciplina (exemplo ilustrativo):

```
Backlog ──► Em progresso ──► Em revisão ──► CI ──► Merge na main ──► No ar
 espera      trabalho          espera       PT        espera          PT
 3 dias       6 h              1 dia       5 min      2 dias         3 min
            (em 2 dias)       (+ 20 min
                               de revisão)
```

	Valor
Tempo de processamento (6 h + 20 min + 5 min + 3 min)	≈ 6,5 h
Lead time (3 + 2 + 1 + 2 dias)	8 dias = 192 h
Eficiência de fluxo	6,5 ÷ 192 ≈ 3%

Leitura do mapa:

- Em 8 dias, o item recebeu cerca de 6,5 horas de trabalho. O resto foi espera.
- O maior tempo parado, depois de começado, foi entre o CI verde e o merge: 2 dias, sem ninguém responsável por integrar. Esse é o gargalo, sustentado por número.
- Trabalhar mais rápido na etapa "Em progresso" mudaria pouco o lead time. Reduzir as esperas muda muito.

7.3 De onde vêm os tempos

Etapa	Fonte
Backlog → em progresso	data de criação do cartão e o campo Início do quadro (Sprint 1)
Em progresso	campos Início e Fim do cartão; primeiro e último commit do ramo
Em revisão	abertura do PR, primeira revisão, aprovação
CI	duração de cada execução (`gh run list`)
Merge e implantação	`mergedAt` do PR e o registro da plataforma

O comando abaixo lê, para cada PR já integrado, os três instantes que viram os tempos de espera: a abertura, a primeira revisão e o merge.

```
gh pr list --state merged --json number,createdAt,mergedAt,reviews \
  --jq '.[] | {pr: .number, aberto: .createdAt, primeira_revisao: (.reviews[0].submittedAt
        // "nenhuma"), merge: .mergedAt}'
```

Os tempos de espera são os mais fáceis de medir: as datas já estão no GitHub e no quadro. O tempo de processamento é o mais difícil: quase sempre é preciso perguntar a quem fez. Quando for estimado, o documento diz que é estimado, e como.

7.4 Do gargalo à melhoria

A rubrica pede um gargalo sustentado por número e uma melhoria com métrica-alvo:

	Fraco	Forte
Gargalo	"a revisão demora"	"os PRs esperaram, em média, 26 h pela primeira revisão (7 PRs da Sprint 2)"

	Fraco	Forte
Melhoria	"revisar mais rápido"	"revisão no mesmo dia: cada integrante olha os PRs abertos ao fim do dia"
Métrica-alvo	—	"espera pela primeira revisão abaixo de 8 h nas duas semanas seguintes"

- A melhoria é uma mudança no processo, com dono, e não um pedido de esforço.
- A métrica-alvo usa o mesmo número que revelou o gargalo: assim é possível saber se melhorou.
- É a mesma estrutura das ações da retrospectiva: responsável, prazo e como verificar.

8. A entrega da Sprint 2

8.1 O que cada critério pede

Critério	O que mostrar
Pipeline de CI e containerização	workflow com build, testes e lint em todo push e PR; verde na `main`; proteção impedindo merge com falha (com a prova do capítulo 4.3); `docker compose up` subindo o projeto do zero
Métricas DORA	ao menos 3 métricas, com a definição de implantação, os comandos, o período, os valores e a interpretação
Mapeamento de Fluxo de Valor	o mapa do fluxo real, com tempos de processamento e de espera medidos (e a fonte de cada um), a eficiência de fluxo calculada, 1 gargalo com número e 1 melhoria com métrica-alvo
Retrospectiva final	`docs/retrospectiva-02.md`: as ações da retrospectiva 01 conferidas com evidência, a evolução do processo com dados e a avaliação do uso de IA

8.2 Comece a medir já

As métricas e o mapa precisam de histórico, e o histórico de novembro começa agora. O que registrar desde o primeiro dia da sprint:

- cada PR aberto, revisado e integrado (o GitHub registra sozinho);
- cada execução do CI (idem);
- cada implantação, com data (releases, tags ou o log da plataforma);
- as datas de início e de fim de cada cartão no quadro;
- o tempo de processamento de alguns itens, anotado por quem fez, no fim do dia.

9. Exercícios e dúvidas frequentes

Exercícios

1. Escreva o workflow mínimo do projeto do grupo, com build, testes e lint, rodando em push na `main` e em pull request.

2. Proteja a `main` do repositório do grupo e prove que a proteção funciona, com um PR que quebra um teste.
3. Escreva a definição de implantação do projeto e os comandos que contam as implantações da última semana.
4. Calcule, para os PRs da Sprint 1 do grupo, o tempo da abertura ao merge. Qual é a mediana? Algum PR destoa? Por quê?
5. Um item teve 4 h de trabalho e lead time de 5 dias corridos. Qual é a eficiência de fluxo, contando 24 h por dia? E contando 8 h úteis por dia? Qual das duas o grupo vai usar, e por quê?
6. Escolha três cartões concluídos e desenhe o mapa do fluxo de cada um, com a fonte de cada tempo.

Respostas do exercício 5: com 24 h por dia, o lead time é 120 h, e a eficiência é 4 ÷ 120 ≈ 3,3%. Com 8 h por dia, o lead time é 40 h, e a eficiência é 4 ÷ 40 = 10%. As duas formas são aceitáveis, desde que o documento diga qual foi usada e use a mesma em todos os itens.

Dúvidas frequentes

Nosso projeto ainda não está no ar. Como medir a frequência de implantação? Definam implantação como merge na `main` com o CI verde e digam isso no documento. A métrica passa a medir a frequência de integração, que é o que o projeto tem.

Podemos usar uma ferramenta que calcula as métricas sozinha? Podem, desde que o método esteja descrito: qual ferramenta, com qual configuração, sobre qual período. A rubrica cobra um método que outra pessoa consiga repetir.

O CI precisa rodar testes de todos os componentes? Precisa rodar os testes que o projeto tem, em todo push e PR. Se o projeto tem mais de um componente, o CI cobre todos.

Um item que voltou de "Em revisão" para "Em progresso" entra no mapa? Entra, e é informação valiosa: o retrabalho aparece como uma volta no fluxo. Contem as voltas e o tempo gasto nelas.

Qual é uma eficiência de fluxo boa? Não há um número certo. Em trabalho de software, é comum a espera dominar o lead time. O que importa é o grupo entender onde está a espera e mostrar a mudança depois da melhoria.

Referências

Todas as páginas foram consultadas em outubro de 2026.

DevOps e integração contínua

- Atlassian — CALMS framework: https://www.atlassian.com/devops/frameworks/calms-framework

- IT Revolution — The Three Ways: https://itrevolution.com/articles/the-three-ways-principles-underpinning-devops/
- Martin Fowler — Continuous Integration: https://martinfowler.com/articles/continuousIntegration.html
- DORA — Continuous integration: https://dora.dev/capabilities/continuous-integration/
- GitHub Actions — Quickstart: https://docs.github.com/en/actions/get-started/quickstart
- GitHub — About protected branches: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches

Containers

- Docker — What is a container?: https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/
- Docker Compose: https://docs.docker.com/compose/

Métricas e fluxo de valor

- DORA — Software delivery performance metrics: https://dora.dev/guides/dora-metrics/
- Lean Enterprise Institute — Value-stream mapping: https://www.lean.org/lexicon-terms/value-stream-mapping/
- GitHub CLI — gh run list: https://cli.github.com/manual/gh_run_list
- GitHub CLI — gh pr list: https://cli.github.com/manual/gh_pr_list